

UNIVERSITÀ DEGLI STUDI DI UDINE

DIPARTIMENTO DI SCIENZE MATEMATICHE,
INFORMATICHE E FISICHE

IBML - Internet of Things, Big Data, Machine Learning



Internet of Things Progetto Pet Tracker

Autori

Andrea Gioia, 169484

Alessio Nicodemo, 166972

Mattia Nadal, 167372

Francesco Sabot, 167339

169484@spes.uniud.it

166972@spes.uniud.it

167372@spes.uniud.it

167339@spes.uniud.it

Sommario

Il progetto Pet-Tracker propone la realizzazione di un sistema IoT per il tracciamento in tempo reale di animali domestici di media taglia. La soluzione integra un dispositivo embedded basato su ESP32-S3 con connettività LTE e modulo GNSS, in grado di acquisire e trasmettere dati di posizione, stato della batteria e attività motoria verso un backend remoto.

L'architettura segue il modello IoT World Forum (IoTWF) e adotta tecniche di ottimizzazione energetica, tra cui modalità Deep Sleep e strategie di Hot Start per il GPS. I dati vengono elaborati e resi disponibili tramite API REST e visualizzati attraverso un'applicazione mobile dedicata.

L'obiettivo principale è garantire un sistema affidabile, a basso consumo energetico e scalabile per il monitoraggio continuo dell'animale.

Indice

1	Architettura del sistema	3
1.1	Mappatura sul modello IoTWF	3
2	Hardware	3
2.1	LiLyGo T-SIM7670G-S3	3
2.2	Antenna GPS e LTE	3
2.3	Batteria e Power Management	3
2.4	BMA400 CJMCU-400	4
3	Firmware: Logica di funzionamento	4
3.1	Gestione della Persistenza e Hot Start	4
3.2	Protocollo di Comunicazione HTTPS	4
3.3	Risparmio Energetico e Deep Sleep	5
3.4	Monitoraggio Hardware e Batteria	5
4	Back-end e Database	6
4.1	Architettura di Hosting e Network Exposure	6
4.2	Sistema di Backup e Data Retention	6
4.3	Monitoraggio e Business Continuity	6
4.4	Struttura dei dati	6
4.5	Interfaccia API e API Rules	7
4.5.1	API Rules e Sicurezza	7
4.6	Smistamento e Normalizzazione	7
4.6.1	Funzionamento del Trigger onRecordAfterCreateSuccess	7
4.6.2	Vantaggi Architettureali e Corrispondenza IoTWF	8
5	Front-end e Applicazione Mobile	8
5.1	Architettura a livelli	8
5.2	Autenticazione e persistenza della sessione	9
5.3	Splash Screen e caricamento parallelo	9
5.4	Comunicazione in tempo reale	9
5.5	Geofencing lato client	10
5.6	Notifiche push	10
5.7	Funzionalità principali	10
6	Test e Risultati	11
7	Limiti del sistema	11
8	Conclusioni	11

1 Architettura del sistema

L'architettura del sistema Pet-Tracker segue il **Modello a 7 livelli dell'IoT World Forum (IoTWF)**, garantendo una transizione fluida dal dato fisico all'informazione per l'utente.

1.1 Mappatura sul modello IOTWF

- **Livello 1 (Physical Devices):** Rappresentato dalla board LilyGo e dai sensori (GPS, Accelerometro).
- **Livello 2 (Connectivity):** Gestito tramite il modem LTE e protocolli di trasporto sicuri.
- **Livello 3 (Edge Computing):** Elaborazione locale sul microcontrollore per il filtraggio delle stringhe NMEA e il calcolo dei passi.
- **Livello 4 (Data Accumulation):** Persistenza dei dati sul server PocketBase (database SQLite).
- **Livello 5 (Data Abstraction):** Esposizione dei dati tramite API REST strutturate in formato JSON.
- **Livello 6 (Application):** L'applicazione mobile dedicata che trasforma le coordinate in posizioni su mappa.
- **Livello 7 (Collaboration and Processes):** L'interazione del proprietario con il sistema per il monitoraggio dell'animale.

2 Hardware

2.1 LiLyGo T-SIM7670G-S3

Il dispositivo utilizza una board LiLyGo T-SIM7670G-S3 [**lilygo_specs**] che si basa sul modulo **ESP32-S3** dual-core, che gestisce sia la logica applicativa che la comunicazione con il modem SIM7670G tramite interfaccia UART.

2.2 Antenna GPS e LTE

Il sistema utilizza un'antenna flessibile LTE [**LTE**] per la connettività dati e un'antenna ceramica attiva [**GPS**] per il ricevitore GNSS integrato, essenziale per la precisione della localizzazione outdoor.

2.3 Batteria e Power Management

Viene impiegata una batteria ricaricabile 3000mAh 30A Litio-Ion ad alta capacità. Il monitoraggio avviene tramite un partitore di tensione collegato al pin **ADC_BAT** (GPIO 4), permettendo di calcolare la percentuale di carica residua.

2.4 BMA400 CJMCU-400

L'accelerometro usato è un **BMA400** [bma400] integrato nella breadboard **CJMCU-400**, che viene interrogato per rilevare il movimento, consentendo l'implementazione di un contapassi utile al monitoraggio della salute dell'animale.

3 Firmware: Logica di funzionamento

Il firmware, sviluppato in ambiente **Arduino** per il SoC **ESP32-S3**, gestisce l'acquisizione dei dati dai sensori, il controllo del modem **LTE** e l'ottimizzazione energetica tramite una macchina a stati finiti.

3.1 Gestione della Persistenza e Hot Start

Per ridurre drasticamente il *Time To First Fix* (TTFF) del modulo GNSS, il firmware sfrutta la memoria **RTC (Real-Time Clock)** dell'ESP32. Questa memoria permette di mantenere i dati vitali anche durante i cicli di *Deep Sleep*. Le variabili critiche, come l'ultima posizione valida (`lastLat`, `lastLon`) e i timestamp (`lastGpsDate`, `lastGpsTime`), vengono dichiarate con l'attributo `RTC_DATA_ATTR`.

Al risveglio del dispositivo, la funzione `iniettaGps()` esegue le seguenti operazioni:

- Recupera le coordinate geografiche e temporali salvate nel ciclo precedente.
- Inietta la posizione nota nel modulo GNSS tramite il comando `AT+CGNSSPOS`.
- Sincronizza l'orario del modem con il comando `AT+CGNSSTIME`.

Questo meccanismo abilita la modalità **Hot Start**, che minimizza il tempo di ricerca dei satelliti e, di conseguenza, il consumo energetico.

```
1 void iniettaGps() {
2   if (!hasGpsFix) return;
3   // Costruzione comando per Hot Start basato su ultima posizione nota
4   String cmdPos = "AT+CGNSSPOS=" + formatCoordinate(lastLat, true) + ","
5     ↪ + latDir + "," +
6     ↪ formatCoordinate(lastLon, false) + "," + lonDir + "
7     ↪ ,0,100";
8   sendAT(cmdPos.c_str());
9 }
```

Listing 1: Ottimizzazione del fix GPS tramite iniezione di dati persistenti

3.2 Protocollo di Comunicazione HTTPS

La trasmissione dei dati al backend PocketBase è affidata a una serie di comandi AT inviati via seriale al modem SIM7670G. Il firmware incapsula le informazioni in un oggetto JSON strutturato, inviato tramite una richiesta **HTTPS POST**. Il payload include:

- `board_id`: un identificativo univoco derivato dall'IMEI del modem;
- `geo`: un oggetto con latitudine e longitudine per la gestione cartografica;
- `battery` / `steps`: telemetria sullo stato di carica e sull'attività motoria rilevata;

- **sleep**: un flag booleano che segnala al server l'imminente entrata in stato di inattività.

```

1 void inviaDati(float l_lat, float l_lon, const BatInfo& bat, const
  ↳ String& timestamp, const StepData& step, bool isSleeping) {
2   String json = "{";
3   json += "\"timestamp\":" + timestamp + "\",";
4   json += "\"geo\":{\"lon\":" + String(l_lon, 6) + "\",\"lat\":" + String(
  ↳ l_lat, 6) + "\",";
5   json += "\"battery\":" + String(bat.voltage, 2) + ",";
6   json += "\"sleep\":" + String(isSleeping ? "true" : "false");
7   json += "}";
8   // Invio tramite comandi AT HTTP
9   modem.print(json);
10 }

```

Listing 2: Esempio di strutturazione del payload JSON

3.3 Risparmio Energetico e Deep Sleep

Il sistema è progettato per massimizzare l'autonomia della batteria da 3000mAh. La logica di risparmio energetico è coordinata con l'accelerometro **BMA400**:

1. **Rilevamento Attività**: Il firmware monitora costantemente il numero di passi. Ogni nuovo movimento resetta un timer di inattività (`lastActivityTime`).
2. **Invio Finale**: Se non viene rilevata attività per un tempo superiore a `SLEEP_TIMEOUT` (30 secondi), il sistema invia un ultimo pacchetto dati aggiornando il flag `sleep` a `true`.
3. **Ibernazione**: Il microcontrollore entra in *Deep Sleep*, spegnendo i moduli radio. Il risveglio è gestito tramite interrupt hardware sul pin `GPIO_5`, generato dal BMA400 solo al rilevamento di un nuovo movimento.

```

1 void enterDeepSleep() {
2   Serial.println("\nEntro in Deep Sleep...");
3   // Configura il pin di risveglio (GPIO 5) collegato al BMA400
4   pinMode(WAKEUP_PIN, INPUT_PULLDOWN);
5   esp_sleep_enable_ext0_wakeup(WAKEUP_PIN, 1);
6   esp_deep_sleep_start();
7 }

```

Listing 3: Configurazione del Deep Sleep con risveglio tramite accelerometro

3.4 Monitoraggio Hardware e Batteria

Il firmware gestisce in modo intelligente l'alimentazione per evitare letture errate o consumi parassiti:

- **Controllo ADC**: Il circuito del partitore di tensione per la batteria viene attivato solo durante la lettura tramite il pin `BAT_ADC_EN`, garantendo la precisione del voltaggio acquisito dal pin `ADC_BAT`.

- **Stato USB:** Attraverso la lettura dei registri hardware dell'ESP32, il sistema rileva se il dispositivo è collegato a una fonte di ricarica USB, settando dinamicamente il campo `charging` nel pacchetto dati.

4 Back-end e Database

Il centro nevralgico della gestione dati è basato su **PocketBase** [pocketbase], una soluzione open-source che integra un database embedded SQLite con un'interfaccia API REST.

4.1 Architettura di Hosting e Network Exposure

L'istanza di PocketBase è stata ospitata su una workstation locale e data la natura del sistema IoT, che richiede la raggiungibilità del backend da parte del dispositivo (tramite rete LTE) e dell'applicazione mobile (tramite rete dati/Wi-Fi), si è reso necessario esporre il servizio locale su rete pubblica. Il problema del *NAT Traversal* e dell'assenza di un indirizzo IP pubblico statico sulla rete locale è stato risolto mediante l'impiego di **ngrok** [ngrok]. Questa tecnologia di *tunneling* crea un tunnel sicuro (HTTPS) tra l'endpoint pubblico di ngrok e la porta locale su cui è in esecuzione il server. Tale URL funge da *Reverse Proxy*, reindirizzando il traffico HTTPS in ingresso verso l'istanza locale di PocketBase, garantendo così la persistenza dei dati e l'accessibilità dei servizi in tempo reale ovunque sia presente una connessione internet.

4.2 Sistema di Backup e Data Retention

Per garantire la resilienza dei dati e prevenire perdite accidentali dovute a guasti del server locale, è stato implementato un sistema di backup automatizzato. Il processo utilizza l'utility **rclone** [rclone], uno strumento a riga di comando per la gestione del cloud storage, configurato per eseguire la sincronizzazione della directory dei dati di PocketBase verso un bucket remoto su **Backblaze B2** [backblaze]. L'automazione è gestita tramite un *cron job* che esegue il backup in modo incrementale ogni primo giorno del mese.

4.3 Monitoraggio e Business Continuity

Per garantire l'affidabilità dell'intero ecosistema, è stato implementato un sistema di monitoraggio esterno tramite **UptimeRobot** [uptimerobot]. Dato che l'accessibilità del backend dipende dalla stabilità del tunnel ngrok, il monitoraggio interroga l'endpoint HTTPS a intervalli regolari, inviando notifiche immediate in caso di downtime.

4.4 Struttura dei dati

I dati ricevuti vengono normalizzati e memorizzati in una collezione denominata `data_sent_raw`. La struttura del record prevede i seguenti campi:

- `timestamp`: riferimento temporale dell'acquisizione.
- `lat / lon`: coordinate geografiche acquisite dal modulo GNSS;
- `battery`: stato di carica della batteria (espresso in voltaggio);

- **battery_percent**: stato di carica della batteria (espresso in percentuale);
- **charging**: booleano che indica se la batteria è in corso di ricarica;
- **steps**: conteggio dei passi rilevato dall'accelerometro BMA400;
- **sleep**: booleano che indica se il dispositivo è in stato di "sleep" (assenza di movimento prolungata);

4.5 Interfaccia API e API Rules

L'accesso ai dati memorizzati nel backend avviene esclusivamente tramite un'interfaccia **RESTful API** fornita nativamente da PocketBase. Questo approccio permette di separare nettamente il livello di persistenza (Database) dal livello di fruizione (Applicazione Mobile).

4.5.1 API Rules e Sicurezza

Per garantire l'integrità e la riservatezza dei dati, sono state configurate specifiche **API Rules** basate su espressioni booleane:

- **List/View**: Visualizzazione permessa solo agli utenti autenticati.
- **Create**: Abilitata per il dispositivo LilyGo tramite autenticazione dedicata.
- **Update/Delete**: Ristrette ai soli amministratori del sistema.

4.6 Logica di Smistamento e Normalizzazione (Server-side Hooks)

Per ottimizzare la gestione dei dati e garantire la separazione delle responsabilità, è stata implementata una logica di smistamento automatico lato server tramite i *JavaScript Event Hooks* di PocketBase. Questa architettura permette di mantenere la collezione `data_sent_raw` come un punto di accumulo "grezzo", delegando al server il compito di distribuire le informazioni nelle tabelle specializzate.

4.6.1 Funzionamento del Trigger `onRecordAfterCreateSuccess`

Il sistema sfrutta l'hook `onRecordAfterCreateSuccess`, che si attiva immediatamente dopo il salvataggio di un nuovo record inviato dalla board LilyGo. Il processo opera su tre direttrici principali:

- **Persistenza Geografica ed Energetica**: I parametri di tensione, percentuale di carica e coordinate GPS vengono estratti e duplicati nelle collezioni `battery_data` e `positions_duplicate`. L'uso di blocchi *try-catch* indipendenti garantisce che un errore in un sensore secondario non comprometta il tracciamento GPS.
- **Logica di Sessione (Activity Tracking)**: Il trigger implementa una macchina a stati basata sul parametro `sleep`. Se il dispositivo è attivo, il sistema verifica l'esistenza di una sessione: in caso positivo aggiorna il conteggio passi, in caso negativo crea una nuova attività.
- **Gestione dello Stato Sleep**: Al ricezione di un pacchetto con `sleep: true`, il trigger chiude la sessione corrente impostando `is_active = false`, permettendo una segmentazione precisa dei periodi di movimento.


```

1 onRecordAfterCreateSuccess((e) => {
2     const raw = e.record;
3     const boardId = raw.getString("board_id");
4     const timestamp = raw.getString("timestamp");
5     const sleep = raw.getBool("sleep");
6     const steps = raw.getInt("steps");
7
8     // 1. Logica di gestione delle sessioni (Activity Tracking)
9     if (!sleep) {
10         if (activeActivity) {
11             // Aggiorna passi e fine sessione corrente
12             activeActivity.set("total_steps", currentSteps + steps);
13             activeActivity.set("end_time", timestamp);
14             e.app.save(activeActivity);
15         } else {
16             // Crea nuova sessione attiva se non esistente
17             activitiesRecord.set("is_active", true);
18             activitiesRecord.set("total_steps", steps);
19             e.app.save(activitiesRecord);
20         }
21     } else if (activeActivity) {
22         // Chiusura sessione per stato SLEEP del dispositivo
23         activeActivity.set("is_active", false);
24         e.app.save(activeActivity);
25     }
26     e.next();
27 }, "data_sent_raw");

```

Listing 4: Implementazione della logica di sessione in PocketBase

4.6.2 Vantaggi Architetture e Corrispondenza IoTWF

Questa implementazione riflette i principi del Modello IoTWF in due punti chiave:

1. **Efficienza del Livello 3 (Edge):** La LilyGo invia un unico pacchetto “bulk”, riducendo il tempo di attività del modem LTE e preservando la batteria.
2. **Ottimizzazione del Livello 5 (Data Abstraction):** L'applicazione Flutter non deve processare dati grezzi; interroga API già normalizzate dal trigger, garantendo una visualizzazione fluida della cronologia.

5 Front-end e Applicazione Mobile

L'applicazione mobile è stata sviluppata in Flutter [flutter], un framework cross-platform che permette di produrre un'unica codebase per Android e iOS. La comunicazione con il backend avviene tramite il PocketBase SDK ufficiale per Dart, che gestisce internamente le chiamate REST e i meccanismi di autenticazione.

5.1 Architettura a livelli

Il codice dell'applicazione è organizzato secondo una separazione netta tra livelli di responsabilità:

- **Services:** logica trasversale e di sistema, indipendente dalla UI (autenticazione, GPS, geocoding, notifiche push).
- **Repositories:** accesso ai dati remoti tramite PocketBase SDK, con esposizione di metodi asincroni e stream tipizzati verso le schermate.
- **Screens:** interfaccia utente che si limita a richiedere i dati ai repository e a reagire ai loro aggiornamenti.

Questa architettura facilita la testabilità e disaccoppia la logica di business dalla presentazione, in linea con il principio di separazione delle responsabilità.

5.2 Autenticazione e persistenza della sessione

L'autenticazione è gestita tramite un `SecureAuthStore` personalizzato che estende l'`AuthStore` nativo di PocketBase. Il token JWT restituito dal server al momento del login viene serializzato in JSON e salvato in modo cifrato tramite `flutter_secure_storage`, che su Android si appoggia al sistema Keystore e su iOS al Keychain.

Al riavvio dell'app, il token viene riletto dallo storage sicuro e viene tentato un refresh automatico della sessione. In caso di successo, l'utente accede direttamente alla schermata principale senza dover reinserire le credenziali; in caso di token scaduto, lo store viene svuotato e l'utente viene reindirizzato al login.

5.3 Splash Screen e caricamento parallelo

La schermata di avvio svolge un ruolo attivo nel precaricare i dati necessari alla home page. Una volta verificata l'autenticazione, vengono avviate in parallelo, tramite `Future.wait`, le seguenti operazioni:

- recupero del profilo utente e dello stato dell'allarme;
- recupero dell'ultima posizione GPS registrata dal dispositivo;
- recupero delle attività del giorno corrente filtrate per `board_id`.

Il `board_id`, identificativo univoco della board LilyGo associata all'utente, viene risolto dinamicamente interrogando la collezione `boards` sul backend, evitando qualsiasi valore hardcoded. Contestualmente, viene calcolata la zona di geofencing in cui si trova l'animale. I dati precaricati vengono poi passati direttamente alla schermata principale, eliminando ulteriori chiamate di rete al momento della navigazione.

5.4 Comunicazione in tempo reale

Per le funzionalità che richiedono aggiornamenti continui — posizione GPS e stato della batteria — l'app sfrutta il meccanismo di *subscribe* offerto da PocketBase, basato su Server-Sent Events (SSE). I repository espongono uno `Stream` tipizzato (rispettivamente `Stream<Positions>` e `Stream<BatteryData>`) a cui le schermate si sottoscrivono tramite `StreamSubscription`. Ogni nuovo record inserito nel backend viene notificato istantaneamente all'app senza necessità di polling.

5.5 Geofencing lato client

La logica di geofencing è implementata interamente lato client tramite l'algoritmo *Ray-Casting*. Dato un punto GPS (latitudine, longitudine) e un poligono definito da una lista di vertici, l'algoritmo traccia un raggio orizzontale dal punto verso l'infinito e conta le intersezioni con i lati del poligono: un numero dispari indica che il punto è interno al poligono.

```
1 static bool isPointInsidePolygon(LatLng point, List<LatLng> polygon) {
2     bool isInside = false;
3     int j = polygon.length - 1;
4     for (int i = 0; i < polygon.length; i++) {
5         if (((polygon[i].latitude > point.latitude) !=
6             (polygon[j].latitude > point.latitude)) &&
7             (point.longitude <
8                 (polygon[j].longitude - polygon[i].longitude) *
9                     (point.latitude - polygon[i].latitude) /
10                     (polygon[j].latitude - polygon[i].latitude) +
11                     polygon[i].longitude)) {
12             isInside = !isInside;
13         }
14         j = i;
15     }
16     return isInside;
17 }
```

Listing 5: Algoritmo Ray-Casting per il geofencing

Le zone vengono recuperate dalla collezione **geofences** su PocketBase, dove ogni record memorizza il nome della zona, i dati dell'indirizzo e la lista dei vertici del poligono. Solo le zone con il flag **is_active** impostato a **true** vengono considerate nel calcolo. Il risultato della verifica è una stringa con il nome della zona di appartenenza, oppure *“Fuori zona sicura”* se il punto non rientra in nessuna area attiva. Questa informazione viene mostrata in tempo reale nella schermata principale.

5.6 Notifiche push

Le notifiche push sono gestite tramite Firebase Cloud Messaging (FCM). Al primo avvio, l'app mostra un dialog personalizzato che spiega all'utente il motivo della richiesta del permesso, prima di invocare la richiesta di sistema. Una volta ottenuto il consenso, il token FCM del dispositivo viene sincronizzato con il profilo utente su PocketBase: il campo **tokenFCM** è modellato come una lista, permettendo all'utente di ricevere notifiche su più dispositivi contemporaneamente. Un controllo di duplicazione evita che lo stesso token venga registrato più volte.

5.7 Funzionalità principali

Mappa in tempo reale. La schermata principale mostra la posizione aggiornata dell'animale su mappa tramite la libreria **flutter_map** con tile OpenStreetMap. La posizione viene aggiornata ad ogni evento ricevuto dallo stream in tempo reale. L'indirizzo corrispondente alle coordinate viene risolto in forma testuale tramite il servizio di reverse geocoding Nominatim.

Riepilogo giornaliero. La schermata di recap, accessibile selezionando una data dal calendario, recupera dal backend le attività e le posizioni GPS relative a quel giorno. Le posizioni vengono rappresentate come una polilinea sulla mappa, restituendo una visualizzazione del percorso effettuato dall'animale. Le statistiche aggregate (passi totali, distanza stimata in chilometri e durata complessiva del movimento) vengono calcolate client-side a partire dai dati ricevuti.

Stato della batteria. La schermata dedicata mostra la percentuale di carica residua tramite un indicatore circolare animato con gradiente. Quando il campo `charging` del record è impostato a `true`, l'indicatore entra in modalità animazione rotante per segnalare visivamente la ricarica in corso. La durata stimata residua viene calcolata proporzionalmente alla capacità nominale della batteria (3000 mAh).

Gestione delle zone sicure. L'app consente di creare, modificare e disattivare le zone di geofencing tramite un editor poligonale interattivo su mappa. L'utente può disegnare il perimetro della zona toccando i vertici direttamente sulla mappa; al salvataggio, le coordinate dei vertici e il centroide del poligono vengono persistiti su PocketBase. L'indirizzo associato alla zona viene ricavato automaticamente tramite geocoding inverso a partire dal centroide.

6 Test e Risultati

7 Limiti del sistema

Nonostante i risultati ottenuti, il sistema presenta alcune limitazioni:

- **Dipendenza dalla rete LTE:** in assenza di copertura, la trasmissione dei dati non è garantita.
- **Prestazioni indoor:** il modulo GNSS risulta poco affidabile in ambienti chiusi.
- **Backend locale:** l'utilizzo di PocketBase su macchina locale, esposto tramite ngrok, introduce dipendenze da servizi esterni e limita la scalabilità.
- **Consumo energetico del modem LTE:** la trasmissione dati rappresenta la principale fonte di consumo.

8 Conclusioni